Museum Heist

Grid-based stealth game

Greedy · Backtracking · Linked List · BST

Nelson de Jesús Navarro De La Rosa
Henry Samuel Garrido Medina · Josué Urrego López
Systems Engineering — Universidad Distrital FJC
Computer Science I · 2026-I

Agenda

- Introduction & goal
- System architecture
- C++ engine & JSON bridge
- Data structures
- Algorithms (Greedy / BT)
- Interface & mechanics
- Testing & validation
- Conclusions

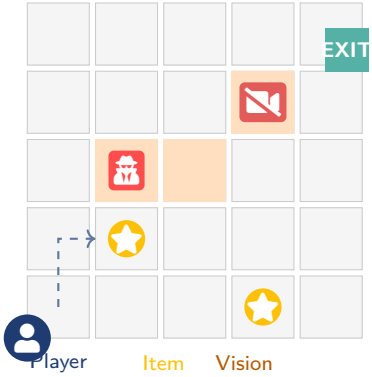
Tech stack: C++ Python JSON Pygame

What is Museum Heist?

Concept

A grid-based stealth game where **every game mechanic is a course algorithm or data structure**.

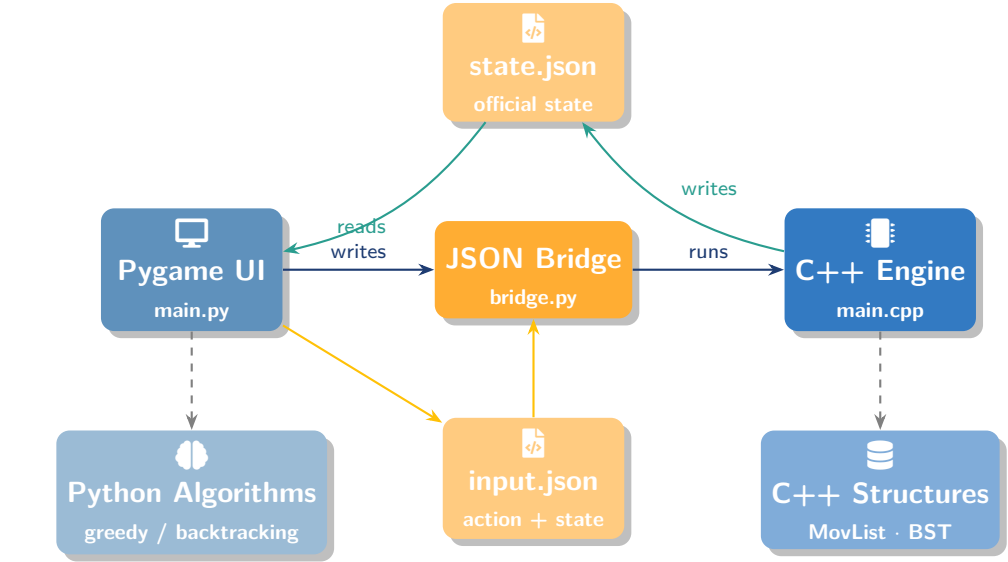
- The player navigates a museum and **collects valuable items**
- Avoids **cameras, guards** and vision zones
- Reaches the **exit** before losing by alarm, movement limit, or time limit



Motivation

Algorithms are not isolated theory — here they

System Architecture



Execution Cycle

- **Input:** Pygame detects a key press (WASD / arrows)
- **Write:** Python writes action + previous state to `input.json`
- **C++ validation:** engine reads the action, checks boundaries, walls, vision zones, items, and writes `state.json`
- **Read:** Python loads the new state
- **Recompute:** greedy and backtracking recalculate
- **Render:** Pygame redraws the board and HUD

[Suggested image]: Screenshot of the game in the playing state, showing the full board and sidebar HUD.

C++ Engine — Difficulty Levels

Parameters by difficulty

Level	Grid	Alarm	Moves / Time
Easy	8×8	4	45 / 90 s
Normal	8×8	3	35 / 60 s
Hard	10×10	2	28 / 45 s

- Compiled with `g++ -std=c++17`
- Validates: bounds, walls, cameras, guards, vision zones
- Wall collision → alarm ++

Listing 1: Movement validation (simplified)

```
// Check destination cell
if (outOfBounds(nr, nc))
  return; // no change

if (grid[nr][nc] == WALL) {
  alarm++;  // wall: alarm
  return;
}
if (isVisionZone(nr,nc)
    || isCamera(nr,nc)
    || isGuard(nr,nc)) {
  game_over = true;
  return;
}
// Valid move
player = {nr, nc};
```

JSON Bridge

input.json (Python → C++)

```
{
  "action": "move",
  "direction": "right",
  "difficulty": "Normal",
  "player": {"row":0,"col":0},
  "score": 0,
  "alarm": 0,
  "elapsed_seconds": 12
}
```

state.json (C++ → Python)

```
{
  "grid": [...],
  "player": {"row":0,"col":1},
  "vision_zones": [...],
  "items_sorted_by_value":
     [...],
  "movement_history": [...],
  "alarm": 0,
  "status": "playing",
  "rank": "In Progress"
}
```

- JSON is human-readable, easy to debug, and available in Python via the standard

Data Structures in C++

⚠ The project does **not** use `std::list`, `std::map`, or `std::set`. Both structures are **implemented from scratch**.

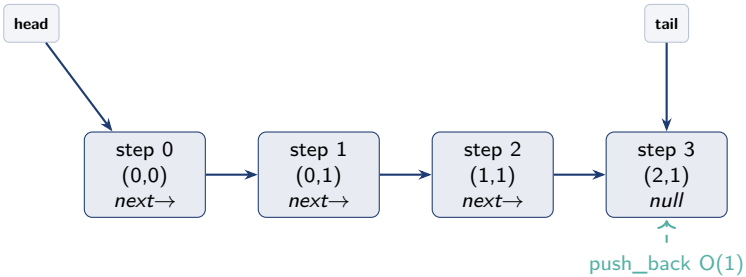
MovementList — Linked List

- Each node stores: `row, col, step, *next`
- `tail` pointer → **append O(1)**
- Traversal to write JSON: O(n)
- Clear: O(n)
- Records **valid movement history**

ItemBST — Binary Search Tree

- Orders items by **value**
- Left → smaller, Right → larger
- Tie-breaker: `item_id`
- Insert: O(log n) avg / O(n) worst case
- Inorder → `items_sorted_by_value`

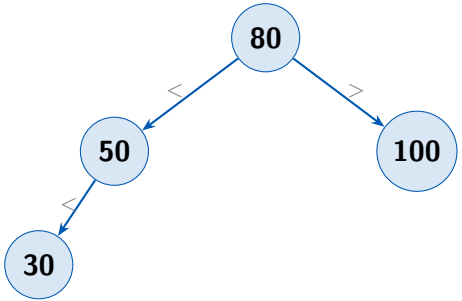
MovementList — Diagram



- **push_back**: always at the end via `tail` → O(1)
- **Traverse**: to serialize JSON → O(n)
- **Clear**: free all nodes → O(n)

[Suggested image]: Screenshot of `linked_list.cpp` showing the `push_back` function.

ItemBST — Diagram



Inorder (ascending): **30** → **50** → **80** → **100**
items_sorted_by_value

[Suggested image]: Screenshot of `tree.cpp` showing the insert function and the inorder traversal.

Greedy Algorithm — Item Selection

Decision logic

1. **Highest-value** item near a guard and **outside** a vision zone
2. If none: highest-value item that is **completely safe**
3. If all are dangerous: highest value with a **warning**

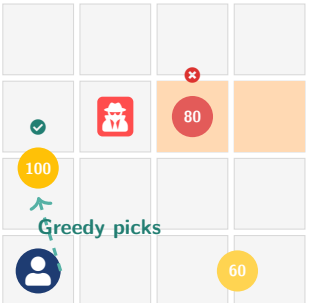
Listing 2: greedy.py (pseudocode)

```
1. Near guard, safe near_guard =
   [i for i in items if near_guard(i) and safe(i)] if near_guard
   return max(near_guard, key = val)
2. Completely safe safe_items =
   [i for i in items if safe(i)] if safe_items :
   return max(safe_items, key = val)
3. Any item (with warning) return
   max(items, key=val)
```

Key property

Makes **one local decision** without exploring future routes. Does **not** guarantee the global optimum.

Greedy — Visualization



- ⭐ **Item 100**: near guard, safe → **chosen**
- ★ **Item 80**: inside vision zone → skipped
- ⭐ **Item 60**: safe but lower value → not selected

📷 **[Suggested image]:** Screenshot of the game with the greedy-selected item highlighted on screen.

Backtracking — Escape Route

Structure: choose – explore – unchoose

1. Add current cell to the route
2. If it is the exit → **success**
3. Explore neighbors ordered by distance to exit
4. If none works → **unchoose** and backtrack

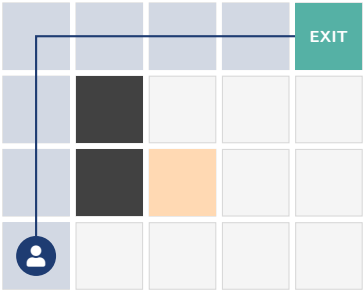
Listing 3: backtracking.py (pseudocode)

```
route.append((row, col))
    visited.add((row, col))
if is_exit(row, col) : return route.copy()
Neighbors by distance for nr, nc in
    neighbors(row,col):  result =
    backtrack(nr, nc) if result:
    return result
Unchoose route.pop()
    visited.remove((row, col)) return
    None
```

Invalid cells:

- Out of bounds
- Walls, cameras, guards

Backtracking — Route Visualization



- **Blue** path = route found by backtracking
- **Avoids** walls (dark), vision zones (orange)
- Updates with **every move**
- Shows whether a feasible route exists

[Suggested image]: Screenshot of the game showing backtracking route markers on the real board.

Algorithm Comparison

Aspect	Greedy	Backtracking
Goal	Choose one item	Find route to exit
Decision	Local / immediate	Recursive / global
Guarantee	Local optimum	Feasible route if it exists
Complexity	$\mathcal{O}(i(g+v))$	$\mathcal{O}(4^n)$ worst case
In practice	Very fast	Bounded by small maps
Language	Python	Python
File	`greedy.py`	`backtracking.py`

Both algorithms are **recomputed every turn** to reflect the current game state. Their output is **visualized directly** on the board and in the HUD.

Interface & Mechanics

Game screens

- **Main menu**: difficulty selection
- **Tutorial**: objective and objects explained
- **Playing**: board + sidebar HUD
- **Caught**: game over with reason
- **Escaped**: final metrics and rank

Controls

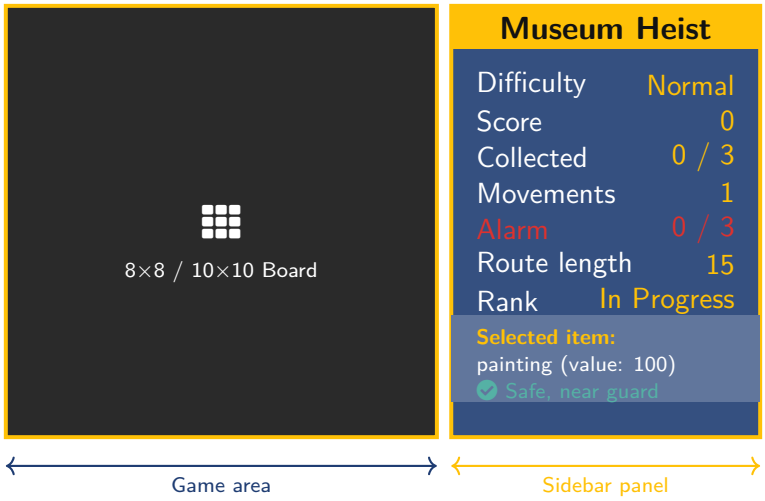
WASD / Arrows	Move player
R	Restart current difficulty
M	Return to main menu

Rendering order

1. Floor (background)
2. Vision zones
3. Backtracking route
4. Walls
5. Items
6. Selected item outline
7. Exit
8. Cameras and guards
9. Player (top layer)

[Suggested image]: Screenshot

HUD — Real-Time Information



Testing & Validation

ID	Test	Expected result
T01	Compilation	Engine builds without errors
T03	Valid move	Player advances 1 cell
T06	Item collection	Score++ and item disappears
T08	Vision zone	Immediate Game Over
T11	Time limit	Game Over by time
T12	Escape	Status = escaped
T15	Greedy	Item + reason returned
T16	Backtracking	Safe route or empty lis

Manual validation

- Select a difficulty
- Navigate the full map
- Intentionally trigger vision zones
- Exhaust alarm, time, and movement limits
- Escape successfully

Limitations & Future Work

Current limitations

- Maps are **hardcoded** (no external files)
- Manual JSON parsing in C++
- BST can become **unbalanced**
- Backtracking optimizes feasibility, **not** score or alarm

Future work

- Maps loaded from **external files**
- Guards with **dynamic patrol routes**
- Balanced BST (AVL / Red-Black)
- Route that optimizes **score + alarm**
- Multiplayer mode

Conclusions

- Course algorithms and data structures become **visible, playable mechanics**
- Architecture separates responsibilities clearly: C++ owns rules, Python owns visualization, JSON connects both
- All requirements are fulfilled: greedy, backtracking, linked list, BST
- Three difficulty levels, win/loss conditions, final metrics, tutorial, and full controls

Museum Heist

Algorithms + Game = Learning

Universidad Distrital FJC
Computer Science I

2026-I

Thank You!

Questions & Discussion